6. Information Extraction with Azure AI Document Intelligence

Diving Deep Into Azure AI
→ Different methods to extract information from the documents using the pre-built models

- ➔ Create a custom AI model and provide feedback to improve the model's accuracy
- ➔ Deploy Azure AI service in the On-premises container

Prerequisites

- ➔ Azure Subscription
- ➔ Docker
- ➔ VS Code with Dev Containers extension
- ➔ Familiarity with:
- ➔ Azure Services
- ➔ Docker
- ➔ Python
- ➔ Jupyter Notebook

Github repository used:

https://github.com/jamesmcroft/document-intelligence-user-feedback-processor

https://github.com/uzi4/document-intelligence-user-feedback-processor - Forked repo

Azure AI Document Intelligence:

- ➔ AI service to build automated data processing software
- ➔ Uses Optical Character Recognition (OCR):
  - Text
  - Key/value pairs
  - Tables
- ➔ Pre-built Models:
  - Invoices
  - Receipts
  - Business cards
- ➔ Azure AI Document Intelligence is a machine learning-based service designed to automate data extraction from documents.

➔ **Pre-built Models**: These models are ready to use for extracting information from common document types like invoices, receipts, ID cards, and business cards.

➔ **Custom Models**: These models are trained on your own specific datasets to extract data from unique forms and documents

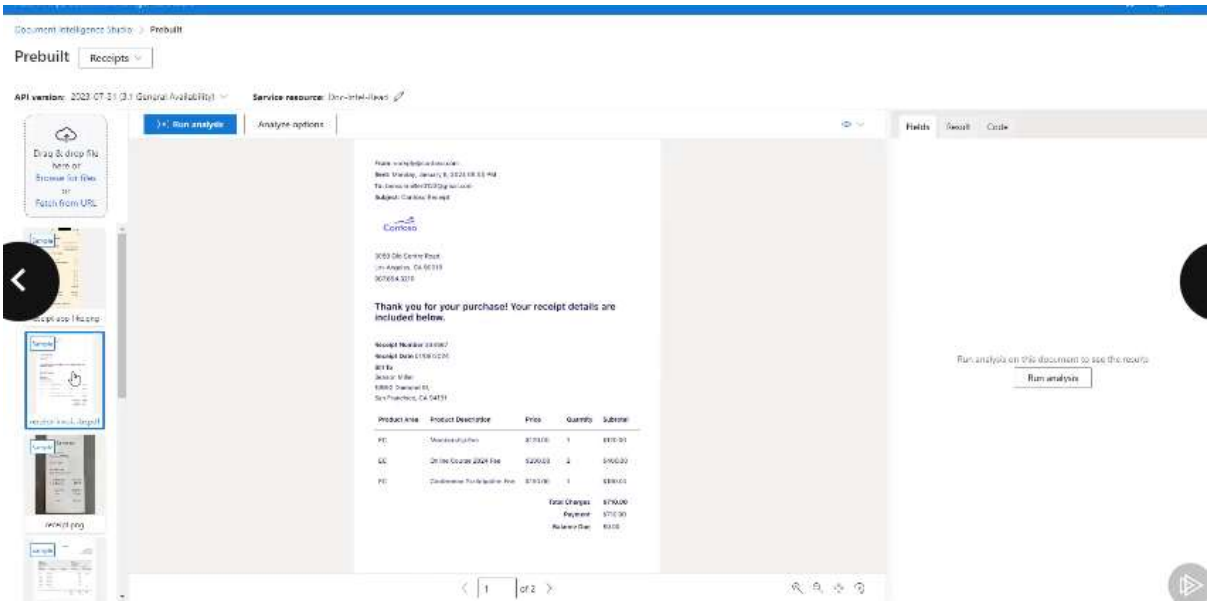
➔ **Service Tools**: You can interact with the service using the Azure AI Document Intelligence Studio for a GUI-based experience or via **SDKs** and REST APIs for programmatic access.

➔ Open Azure Ai website platform

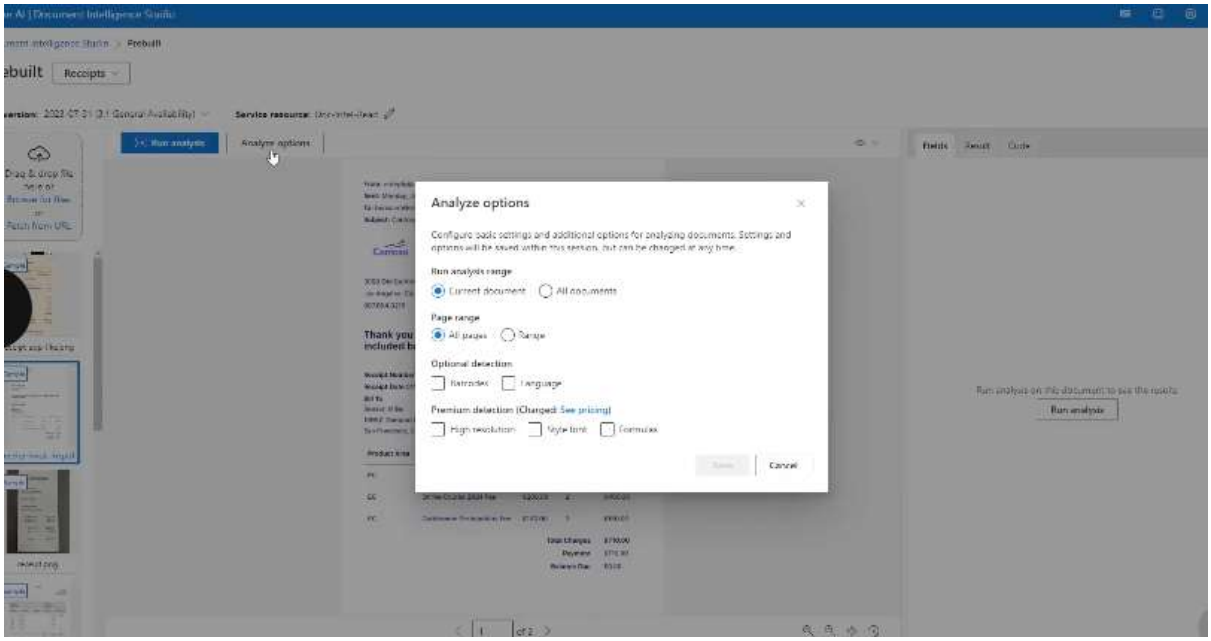
➔ Then create Document intelligence and fill all the required details

➔ Then open document intelligence studio

➔ After opening document intelligence studio and use a prebuilt model for analyzing receipt



➔ Click on analyze option and select analyzing options



➔ And click on run analyses

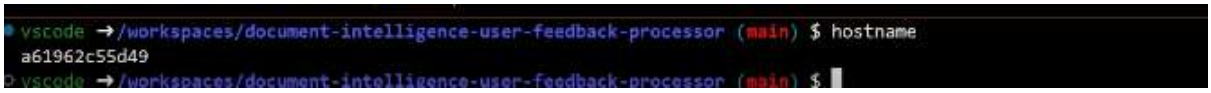
➔ And it will be extract information about the text present on receipt

➔ And code for extract code for extract information on local system

Incorporate Human feedback and Correction into the training Process

➔ Azure Al Document Intelligence Studio custom model
➔ Implement a feedback mechanism to collect feedback and retrain the model

➔ How to run a dev container and setup up required azure ai infrastructure:
➔ Step 1: open command pallete
➔ Step 2: Search Dev container
➔ Step 3: Then select the git repostitory folder where dev container folder is present
➔ Step 4: open terminal and check host name



➔ Step 5: Setting up azure infrastructure

➔ Step 6:Logging to Azure AI account by using dev container using azure cli

```
vscode ➜ /workspaces/document-intelligence-user-feedback-processor (main) $ az login --tenant
```

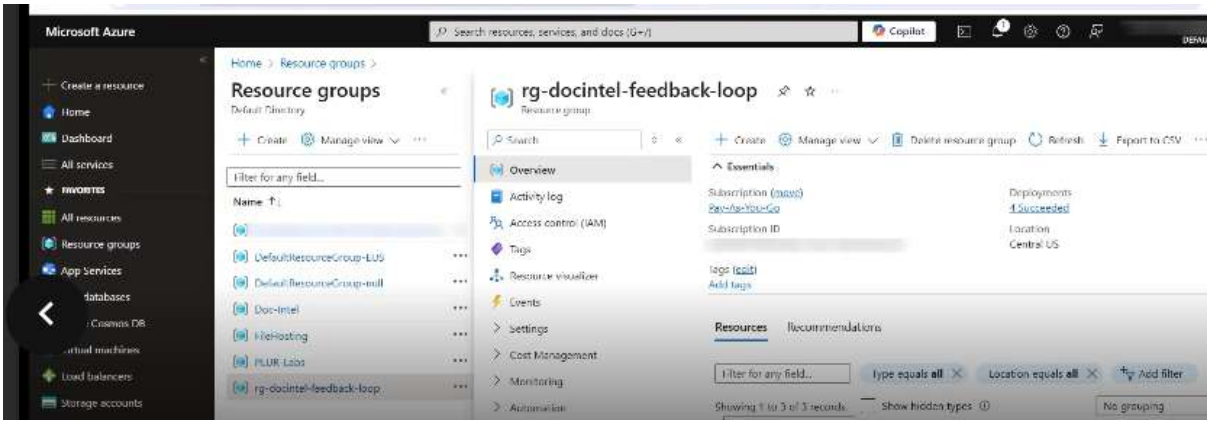
➔ Step 7 : file permission as new files can be created when running

```
vscode ➜ /workspaces/document-intelligence-user-feedback-processor (main) $ sudo chmod -R 777 ../document-intelligence-user-feedback-processor/
```

➔ Step 8: Run a new powershell console and run new setup command for setting , this will start creating the required azure resources

```
PS /workspaces/document-intelligence-user-feedback-processor> ./Setup-Environment.ps1 -DeploymentName docintel-feedback-loop -Location centralus -SkipInfrastructure $true
Starting environment setup...
Deploying infrastructure...
Starting infrastructure deployment...
```

➔ Step 9 : then by using azure ai portal you can see many resources are being created



➔ **Building a custom extraction model :**

1. Import all the modules and libraries

```python
import os

from dotenv import dotenv_values
from azure.identity import DefaultAzureCredential
from modules.app_settings import AppSettings
from modules.model_training_client import ModelTrainingClient
from modules.document_canvas import (DocumentCanvas)
from modules.document_intelligence_label import DocumentIntelligenceLabel
from modules.document_intelligence_result_formatter import DocumentIntelligenceResultFormatter
```

   a. Azure Authentication: The code uses DefaultAzureCredential from azure.identity, which is a common method for handling authentication across different Azure environments securely.
   b. Document Intelligence: It imports components (DocumentIntelligenceLabel, DocumentIntelligenceResultFormatter) related to analyzing and extracting data from documents using Azure AI services.

c. Custom Modules: The code depends on local modules (modules.app_settings, modules.model_training_client) to manage application configurations and handle custom model training logic

2. Then configuring Authentication :

```
working_dir = os.path.abspath('')
settings = AppSettings(dotenv_values(f"{working_dir}/config.env"))
azure_credential = DefaultAzureCredential(
    exclude_environment_credential=True,
    exclude_managed_identity_credential=True,
    exclude_shared_token_cache_credential=True,
    exclude_interactive_browser_credential=True,
    exclude_powershell_credential=True,
    exclude_visual_studio_code_credential=False,
    exclude_cli_credential=False
)
```

a. Credential Source: It uses DefaultAzureCredential, which attempts to authenticate using various methods in a specific order .
b. Excluded Methods: The configuration explicitly disables several methods, including environment variables, managed identities, shared token caches, interactive browser login, and PowerShell credentials.
c. Enabled Methods: It only allows authentication via Visual Studio Code and Azure CLI credentials.
d. Configuration File: The AppSettings object is populated using a .env file located in the current working directory

3. This sets the model name and version number:

```
# The name of the model
model_name = 'invoices'

# The version of the model
initial_model_version = '1.0.0'

# The name of the model that will be registered in Azure AI Document Intelligence
initial_model_id = f"{model_name}-{initial_model_version}"
```

✓ 0.0s

4. Creating model client

```
model_training_client = ModelTrainingClient(settings=settings, use_azure_credential=False, azure_credential=azure_credential)
```

a. Initialization: It sets up the client using a configuration object called settings.
b. Authentication: The parameter use_azure_credential=False suggests it is skipping default automatic authentication methods, such as those relying on DefaultAzureCredential().
c. Credentials: Instead, it explicitly passes an authentication object, azure_credential, to manage permissions for connecting to the Azure ML workspace

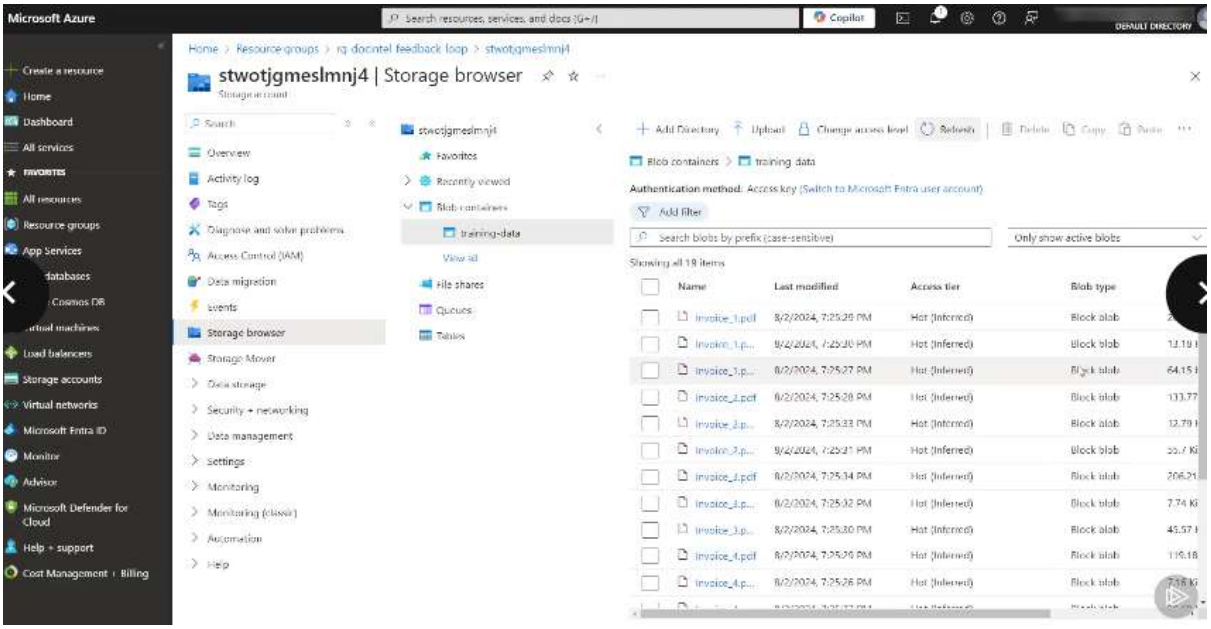
5. Uploads the training data and create model by using intelligence:

```
model_training_client.delete_training_data("Invoice_6")

# Uploads the initial training set to Azure Blob Storage and initiates model training using the upl
model_training_client.upload_training_data(f"{working_dir}/model_training")
invoice_model = model_training_client.create_model(model_name=initial_model_id)
```

   a. delete_training_data: This method is used to remove specific data from a training set in Azure Blob Storage before starting a new training run.
   b. upload_training_data: This line uploads training documents from a local directory (specified by working_dir) to Azure Blob Storage, preparing them for the model training process.
   c. create_model: This command initiates the training of a new machine learning model using the data just uploaded, assigning it a specific initial_model_id.

6. Then you can check azure ai platforms storage browser and be able to see that training data is being uploaded



7. Then setting up the file and storage structure
8. Then analyzing layout of documents when training a custom model in Azure AI document Intelligance
9. Then running a script that Generates corresponding ocr file

```
model_training_client.run_layout_analysis(pdf_path, pdf_feedback_path, initial_model_id)
```

10. Analyses

11. Then displaying the document analyzed by intial training model

```
doc_canvas = DocumentCanvas(working_dir)

canvases = doc_canvas.load_pdf(pdf_path, document_fields_path, pdf_feedback_path)
for canvas in canvases:
    display(canvas)
✓ 1.6s
```

12. Taking human feedback
13. Then once the feedback is being generated , the rendered the information of label entered

```
labels = [DocumentIntelligenceLabel(label_region, doc_canvas.fields) for label_region in doc_canvas.get_document_labels()]
ell (Ctrl+Alt+Enter)
for label in labels:
    display(label.render())
```

    a. doc_canvas.get_document_labels(): This function call likely retrieves a set of labeled regions (bounding boxes or text spans) previously defined for a document.List
    b. Comprehension: The code constructs a list called labels by iterating through each label_region retrieved from the document canvas.
    c. label.render(): This method is called to display the visual representation of each label, which typically means drawing bounding boxes around the identified document fields.

14. Creating json file with updated values

```
DocumentIntelligenceResultFormatter.save_to_labels_json(labels, pdf_file_name, pdf_labels_path)
```

15. Then creating a new training model by using the new updated value that was formatted

```
# The version of the updated model, in this example, a minor change by adding a new training docu
updated_model_version = "1.1.0"

# The name of the model that will be registered in Azure AI Document Intelligence
updated_model_id = f"{model_name}-{updated_model_version}"

# Uploads the updated user feedback documents to Azure Blob Storage and initiates model training
model_training_client.upload_training_data(pdf_dir)
updated_model = model_training_client.create_model(model_name=updated_model_id)
```

    a. Model Versioning: The code defines a new version number (e.g., "1.1.0") and creates a unique model ID by combining the base model name with this new version.
    b. Data Upload: It uses a client to upload training documents (PDFs) from a specified local directory to Azure Blob Storage.
    c. Model Training: The create_model function is then called to initiate the training process on the Azure service using the newly uploaded data

16. Analyse the file with new training model

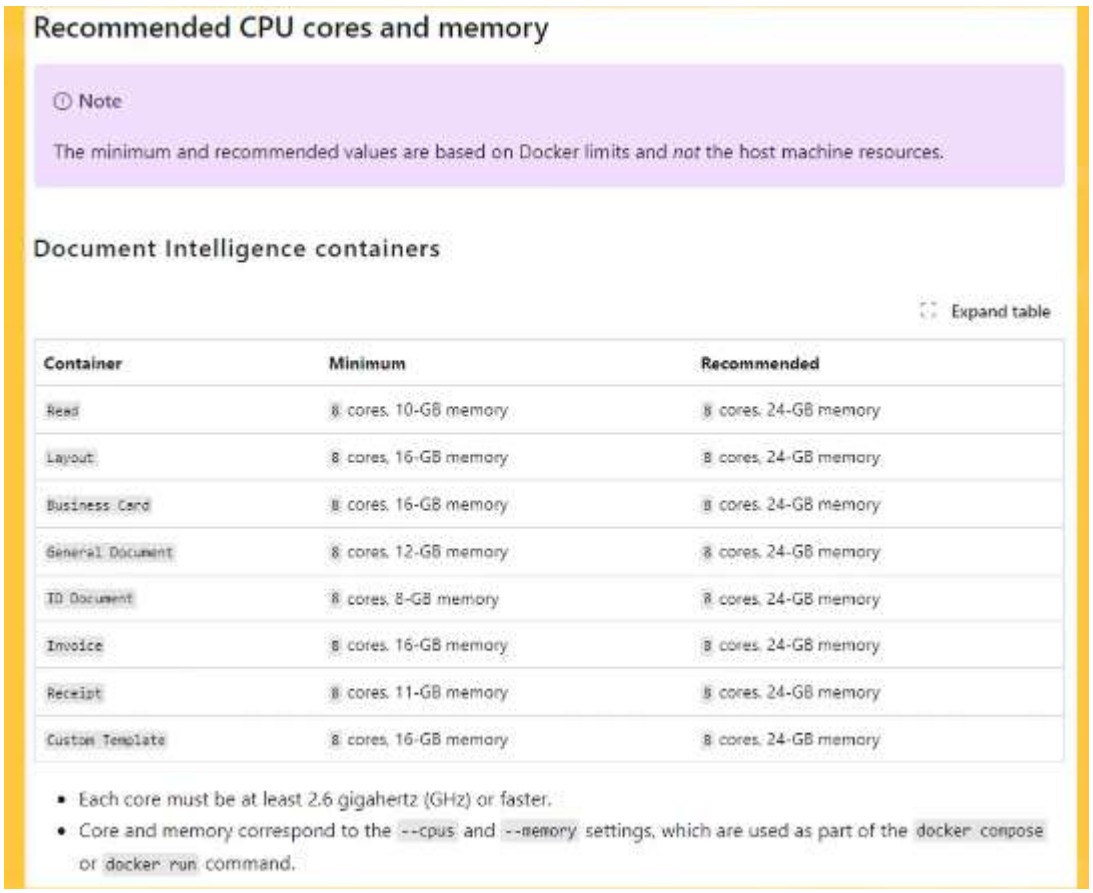
```
 The file path to where the updated analysis of the user feedback document will be stored.
df_updated_analysis_path = os.path.join(pdf_dir, f"{pdf_file_name}.ocr_{updated_model_version}.json")

 Run layout analysis with the updated model
odel_training_client.run_layout_analysis(pdf_path, pdf_updated_analysis_path, updated_model_id)
```

17. The display the result

Deploy Document Intelligence Service on the On-premises Container

➔ Container will allow you to run document intelligence system on your own network



Recommended CPU cores and memory

ⓘ Note

The minimum and recommended values are based on Docker limits and *not* the host machine resources.

Document Intelligence containers

Expand table

Container	Minimum	Recommended
`Read`	`8` cores, 10-GB memory	`8` cores, 24-GB memory
`Layout`	`8` cores, 16-GB memory	`8` cores, 24-GB memory
`Business Card`	`8` cores, 16-GB memory	`8` cores, 24-GB memory
`General Document`	`8` cores, 12-GB memory	`8` cores, 24-GB memory
`ID Document`	`8` cores, 8-GB memory	`8` cores, 24-GB memory
`Invoice`	`8` cores, 16-GB memory	`8` cores, 24-GB memory
`Receipt`	`8` cores, 11-GB memory	`8` cores, 24-GB memory
`Custom Template`	`8` cores, 16-GB memory	`8` cores, 24-GB memory

- Each core must be at least 2.6 gigahertz (GHz) or faster.
- Core and memory correspond to the `--cpus` and `--memory` settings, which are used as part of the `docker compose` or `docker run` command.

➔

➔ You can deploy thes container option on your network

➔ Deploying Read container

➔ Then running docker run command

```
docker run -d \
  --name DocIntelLocal \
  -e EULA=accept \
  -e billing= \
  -e apiKey=  \
  -e AzureCognitiveServiceLayoutHost=http://DocIntelLocal:5000 \
  -p 5000:5000 \
  mcr.microsoft.com/azure-cognitive-services/form-recognizer/read-3.1
```

- Image: It pulls the container image form-recognizer/read-3.1 from the Microsoft Container Registry (MCR).
- EULA: By setting -e EULA=accept, you are agreeing to the End User License Agreement required to run the container.
- Networking: The -p 5000:5000 flag maps port 5000 on your host machine to port 5000 inside the container, allowing you to access the service at http://localhost:5000.
- Billing & Authentication: The -e billing= and -e apiKey= lines are placeholder arguments that must be filled with your specific Azure resource endpoint and API key to track usage and bill your Azure subscription

➔ Setting up services billing endpoints extracting from azure ai platform

```
-e billing=https://di-wotjgmeslmnj4.cognitiveservices.azure.com/ \
-e apiKey=dc6f81687ff745e5adbc7faef906af1a \
```

➔

➔ When docker command is ready then executing it on terminal to start a new container

➔